

Comparison of `dmrpp-schema-narrative.adoc` and `dmrpp.xsd`

This compares `xml-schema/dmrpp-schema-narrative.adoc` against `xml-schema/dmrpp.xsd`, treating `dmrpp.xsd` as the single source of truth for the XML vocabulary, datatypes, required attributes, and content models.

The narrative document includes implementation and semantic detail that XML Schema cannot represent. Those additions are called out separately from schema conflicts.

Executive Summary

The narrative broadly follows the schema's main vocabulary, but it currently has several conflicts that should be corrected before it is treated as a reliable explanation of the schema:

- The narrative examples use an older or different DMR++ namespace URI than the schema.
- Several examples use bracketed comma-separated `chunkPositionInArray` values, while the schema requires a whitespace-separated `xs:list` of unsigned integers.
- The narrative says `dmrpp:chunks` contains exactly one `chunkDimensionSizes`, while the schema currently allows one or more.
- The narrative says `dmrpp:chunks` may contain chunks or blocks, but not both. The schema permits an unbounded choice containing any mix of `chunk` and `block`, including zero of both after `chunkDimensionSizes`.
- The narrative says `dmrpp:chunk` requires `offset` and `size`; the schema requires `offset`, `nBytes`, and `chunkPositionInArray`.
- The narrative describes a `byteOrder` attribute on `dmrpp:chunk`; the schema only defines `byteOrder` on `dmrpp:chunks`.
- The narrative example uses `deflate_level`; the schema defines `deflateLevel`.
- The narrative describes fixed-length string padding values that do not match the schema's `PadKind` enumeration.
- The narrative describes `DIO` as boolean, while the schema defines it as `xs:string`.

Schema Facts

The schema defines the namespace as:

```
http://opendap.org/ns/dmrpp/1.0#
```

The schema version is `1.0`. It uses qualified elements and unqualified attributes.

The schema defines these global elements:

Element	Type	Main content model
<code>dmrpp:chunks</code>	<code>ChunksType</code>	one or more <code>chunkDimensionSizes</code> , then zero or more <code>chunk</code> or <code>block</code> elements
<code>dmrpp:chunkDimensionSizes</code>	<code>IndexList</code>	whitespace-separated unsigned integers
<code>dmrpp:chunk</code>	<code>ChunkType</code>	empty element with required and optional attributes
<code>dmrpp:block</code>	<code>BlockType</code>	empty element with required and optional attributes
<code>dmrpp:FixedLengthStringArray</code>	<code>FixedLengthStringArrayType</code>	empty element with required and optional attributes
<code>dmrpp:compact</code>	<code>CompactType</code>	base64 text content
<code>dmrpp:missingdata</code>	<code>MissingDataType</code>	base64 text content
<code>dmrpp:specialstructuredata</code>	<code>SpecialStructureDataType</code>	base64 text content
<code>dmrpp:vlsa</code>	<code>VlenStringArrayType</code>	zero or more <code>v</code> child elements

The schema also defines the `DatasetExtensionAttributes` attribute group:

Attribute	Type	Use	Default
<code>href</code>	<code>xs:anyURI</code>	required	none
<code>trust</code>	<code>xs:boolean</code>	optional	<code>false</code>
<code>version</code>	<code>xs:string</code>	optional	none

Element-by-Element Comparison

`dmrpp:chunks`

Matches:

- The narrative correctly identifies `dmrpp:chunks` as the container for chunk layout metadata.
- It lists all schema-defined `dmrpp:chunks` attributes: `compressionType`, `deflateLevel`, `fillValue`, `byteOrder`, `structOffset`, `LBChunk`, and `DIO`.
- It correctly states that all `dmrpp:chunks` attributes are optional.
- It adds useful semantics for filters, fill values, structure offsets, linked-block handling, and Direct I/O behavior.

Conflicts with the schema:

- The narrative says there is exactly one `chunkDimensionSizes`. The schema allows `minOccurs="1"` and `maxOccurs="unbounded"`. The schema comment says this is probably a bug and implementation uses the first, but the current XSD still permits more than one.
- The narrative says `dmrpp:chunks` contains either one or more chunks or one or more blocks, but not both. The schema defines an unbounded choice after `chunkDimensionSizes`, so it permits mixed `chunk` and `block` children and also permits zero chunks/blocks.
- The narrative says `DIO` is boolean. The schema defines `DIO` as `xs:string` and comments that the implementation looks for `DIO="off"`.
- The narrative says `byteOrder` defaults to `BE`. The schema does not define an XML Schema default. A schema comment says little-endian is assumed by default, with a possible DIO big-endian case.
- The narrative says `dmrpp:chunk` also includes a `byteOrder` attribute. The schema does not define `byteOrder` on `ChunkType`.

`dmrpp:chunkDimensionSizes`

Matches:

- The narrative correctly describes this element as a whitespace-separated list of chunk sizes.
- The narrative correctly says the element has no attributes or child elements.

Conflicts with the schema:

- The narrative says there is exactly one per `dmrpp:chunks`, but the schema currently allows one or more.

Schema nuance:

- The type is `IndexList`, an `xs:list` of `xs:unsignedLong`. That means values must be whitespace-separated unsigned integers. Commas and brackets are not valid lexical values for this type.

`dmrpp:chunk`

Matches:

- The narrative correctly lists schema-defined attributes `offset`, `nBytes`, `chunkPositionInArray`, `fm`, `href`, `trust`, and `LinkedBlockIndex`.
- It correctly says `dmrpp:chunk` has no child elements.
- It adds useful semantic detail for filter masks, inherited/default data URLs, trusted URLs, and linked block indices.

Conflicts with the schema:

- The narrative says each chunk must include `offset` and `size`; the schema requires `offset` and `nBytes`.
- The narrative says `chunkPositionInArray` is required only for variables with more than one chunk. The schema makes it required for every `dmrpp:chunk`.

- The narrative examples use values such as `[0,0]` and `[20,20,20,20]`; the schema's `IndexList` requires whitespace-separated values such as `0 0` and `20 20 20 20`.
- The narrative implies inherited values from `dmrpp:chunks` for chunk attributes. The schema only defines `href` and `trust` directly on `ChunkType`; it does not define `offset`, `nBytes`, `chunkPositionInArray`, `fm`, or `LinkedBlockIndex` inheritance.
- The narrative says the default value of `fm` is `0`. The schema does not define a default for `fm`.

dmrpp:block

Matches:

- The narrative correctly lists `offset`, `nBytes`, `href`, and `trust`, which are all schema-defined attributes.
- It correctly says `dmrpp:block` has no child elements.
- It adds semantic detail about assembling blocks into a buffer.

Potential issue:

- The narrative note says `href` and `trust` might not be supported by `dmrpp:block`, but the schema defines them as valid optional attributes. If the note is about implementation support, it should be framed explicitly as implementation uncertainty, not schema uncertainty.

dmrpp:FixedLengthStringArray

Matches:

- The narrative correctly identifies `string_length` and `pad`.
- It correctly says the element has no child elements.
- It adds useful semantics about slicing raw byte buffers into fixed-length strings.

Conflicts with the schema:

- The schema requires `string_length` as `xs:positiveInteger`; the narrative should state that it is required and positive.
- The schema defines `pad` as optional with default `not_set`; the narrative does not mention the default.
- The narrative gives padding examples `null`, `space`, and `zero`. The schema's allowed values are `null_pad`, `null_term`, `space_pad`, and `not_set`.

dmrpp:compact

Matches:

- The narrative correctly describes base64-encoded inline data.
- It correctly says there are no attributes or child elements.

No schema conflicts found.

dmrpp:missingdata

Matches:

- The narrative correctly describes base64-encoded missing-data payloads.
- The added projection and zlib behavior is semantic implementation detail outside the schema.

Potential omission:

- For consistency with `compact`, the narrative should explicitly say `dmrpp:missingdata` has no attributes and no child elements. The schema defines only simple base64 text content.

`dmrpp:specialstructuredata`

Matches:

- The narrative correctly describes a base64-encoded special structure payload.
- The structure-member limitations and decoding behavior are useful semantic detail outside the schema.

Potential omission:

- The narrative should explicitly say `dmrpp:specialstructuredata` has no attributes and no child elements. The schema defines only simple base64 text content.

`dmrpp:vl` and `dmrpp:v`

Matches:

- The narrative correctly describes `dmrpp:vl` as containing `v` values.
- It correctly identifies the `c` attribute as an optional count for repeated values.
- The example's unprefix `v` children are consistent with the default DAP namespace unless the intended children are in the DMR++ namespace. However, because `elementFormDefault="qualified"` and the local `v` declaration is inside the DMR++ schema, valid `v` children should be in the DMR++ namespace when validated directly against this schema.

Conflicts or clarifications needed:

- The narrative refers to `dmrpp:v`, but the example uses unprefix `<v>` elements under a default DAP namespace. Under the schema namespace rules, that example likely does not validate as written.
- The narrative refers to `dmrpp:c`, but the schema defines `c` as an unqualified attribute because `attributeFormDefault="unqualified"`. The valid attribute is `c`, not `dmrpp:c`.
- The narrative says the example vector has twelve elements with "Parting", "is such", and ten "sweet" values. The example actually declares `<Dim size="10"/>` and values `Bannana`, `bread`, `is`, six `so`, and `tasty!`, which expands to ten values.
- The schema permits `dmrpp:vl` with zero `v` children.

Example-Level Problems

The examples are the highest-risk part of the narrative because readers may copy them.

Namespace URI

The examples use:

```
xmlns:dmrpp="http://xml.opendap.org/dap/dmrpp/1.0.0#"
```

The schema defines:

```
xmlns:dmrpp="http://opendap.org/ns/dmrpp/1.0#"
```

If `dmrpp.xsd` is the source of truth, the examples should use the schema namespace.

`chunkPositionInArray`

The examples use bracketed comma-separated values:

```
chunkPositionInArray="[0,0]"
```

The schema requires whitespace-separated `xs:unsignedLong` values:

```
chunkPositionInArray="0 0"
```

`deflateLevel`

Example 2 uses:

```
<dmrpp:chunks deflate_level="6" compressionType="deflate">
```

The schema defines:

```
<dmrpp:chunks deflateLevel="6" compressionType="deflate">
```

Narrative-Only Semantics Worth Preserving

These statements are not directly represented by XML Schema, but they appear useful and should probably remain in a narrative specification if accurate:

- Filter ordering matters, and filters must be applied in reverse order during decoding.
- `compressionType` may describe non-compression filters such as shuffle and fletcher32.
- `fillValue` supports omitted all-fill chunks and may need structure-member handling.
- `structOffset` describes simple structure layout and total structure size.
- `LBChunk` is an interpreter hint for HDF4 linked block chunks.
- `DIO` describes a Hyrax output optimization rather than an intrinsic data-reader feature.

- `fm` describes HDF5 per-chunk filter masks and affects whether decompression is attempted.
- `href` and `trust` support trusted access to external data resources.
- `FixedLengthStringArray` describes how raw byte buffers become fixed-length strings.
- `missingdata` and `specialstructuredata` decoding behavior includes implementation details beyond base64 lexical validity.
- `vlsa` uses run-length counts to represent repeated variable-length string values compactly.

Recommended Narrative Updates

1. Update all DMR++ namespace examples to <http://opendap.org/ns/dmrpp/1.0#>.
2. Replace bracketed comma-separated `chunkPositionInArray` examples with whitespace-separated unsigned integer lists.
3. Replace `deflate_level` with `deflateLevel`.
4. Replace `size` with `nBytes` when describing required `dmrpp:chunk` attributes.
5. State that `chunkPositionInArray` is schema-required for every `dmrpp:chunk`, then separately explain any implementation tolerance if it exists.
6. Align the `chunkDimensionSizes` cardinality text with the current schema: one or more are schema-valid, although the schema comments say exactly one is intended and only the first is used.
7. Align the `chunk/block` child model with the current schema: the XSD permits zero or more `chunk` or `block` children in any mixture after `chunkDimensionSizes`. If this is unintended, fix the schema rather than only the narrative.
8. Remove the claimed `dmrpp:chunk byteOrder` attribute, or mark it as historical if older DMR++ documents used it.
9. Describe `DIO` as string-valued in the schema and document the implementation's expected values, especially `off`.
10. Update `FixedLengthStringArray` padding examples to `null_pad`, `null_term`, `space_pad`, and `not_set`.
11. Clarify that `v` children and the `c` attribute are governed by the schema namespace rules: `v` is an element in the DMR++ namespace; `c` is an unqualified attribute.
12. Fix the VLSA example prose so the expanded values and declared dimension agree.

Possible Schema Issues Exposed by the Comparison

These are not narrative defects if `dmrpp.xsd` is the source of truth, but they are places where the schema itself may not express the intended model:

- `ChunksType` allows multiple `chunkDimensionSizes` elements even though the schema comment says there should be exactly one.
- `ChunksType` allows arbitrary mixtures of `chunk` and `block` children, even though the narrative says they are mutually exclusive except for linked-block arrangements.
- `ChunksType` allows no `chunk` or `block` children after `chunkDimensionSizes`; the schema comments say this can represent an all-fill array, but the narrative should make that explicit.
- `DeflateLevelList` allows any non-negative integers, while the narrative says valid deflate levels are 1 through 9.
- `DIO` is unconstrained `xs:string`, while the narrative describes boolean behavior and the schema comment identifies `off` as meaningful.

- `compressionType` is unconstrained `xs:string`, while the narrative lists known filters and says order matters.
- `fillValue` is unconstrained `xs:string`, which cannot validate structure-member fill-value arity or lexical type.
- `IndexList` validates unsigned integers but cannot validate rank, chunk-grid shape, or whether `chunkPositionInArray` is within the array's logical chunk space.